
Lab 3: Assembly Language and System Calls Primer

Lab Goals

- Initial introduction to writing simple programs in assembly language.
- To get acquainted with the low-level interface to system calls.
- Calling assembly code from C and vice-versa.
- Basics of directory listings.

This lab may be done in pairs!

As usual, you should read and understand the reading material and complete task 0 before attending the lab. It is also extremely important to have fully absorbed the material from lecture 3, as the lecture contains example code that can be used with minor changes to implement many of the subtasks in this lab.

For the entire lab, do not use the standard library! This means you shouldn't include `stdio.h`, `stdlib.h`, and do not link to any other file from the C standard library. You can, however, include your own files or any files we provide you. This also means that you cannot use any library functions like `printf`, `fopen`, `fgetc`, `strcmp`, etc.

Task 0: Using nasm, ld and the Arguments Printing Program

In this task we will build a program which prints its arguments to standard output without using the standard C library.

Task 0.A: C Implementation

Download the files `main.c`, `start.s`, `util.c` and `util.h`.

Compile and link them without using the C standard library as follows:

Assemble the glue code:

```
nasm -f elf32 start.s -o start.o
```

Compile the `main.c` and `util.c` files into object code files:

```
gcc -m32 -Wall -ansi -c -nostdlib -fno-stack-protector util.c -o util.o
gcc -m32 -Wall -ansi -c -nostdlib -fno-stack-protector main.c -o main.o
```

Link everything together:

```
ld -m elf_i386 start.o main.o util.o -o task0
```

- Write a makefile to perform the compilation steps automatically.
- Write a new `main.c` that prints the elements of `argv` to the standard output, without using `stdlib`. This part is important, as here is where you make sure that you have the compiler set up correctly to work using the **CDECL C calling convention**, as described in class.

Explanation

The file `start.s` has two purposes:

- I Each executable must have an **entry point** — the position in the code where execution starts. By default, the linker sets this entry point to be a library supplied code or function that begins at `_start`. This code is responsible for initializing the program. After initialization, this code passes control to the `main()` function. Since we are not using any standard libraries, we must supply the linker with `_start` of our own — which is defined in `start.s`.
- II The assembly-language source code in `start.s` also contains the `system_call` function, which is used to get a direct system call without requiring you to

write in assembly language. Note that you can link files written in different languages: an object file is an object file, no matter where it came from. All is machine code at some point!

Task 0.B: Assembly Language Primer

Implement a stand-alone program in assembly language that prints a constant string, such as "hello world" and a linefeed to stdout. For this sub-task you need to read about argument passing to a system call in assembly, and look at `start.s` as an example. Your program should not use anything other than Linux system services.

Task 0.C: Recalling Encoder from Lab A

Make sure you have a good implementation in C of the encoder from lab A, in the sense that it has handled the tasks with a good understanding of low-level features. That is because later on in the lab you will need to implement a simplified form in assembly language with direct system calls. A good low-level C implementation will make the task immediate, whereas an implementation without a care for details and understanding of low-level features (e.g. what is the end of a "string"?) will be extremely unhelpful. Consider that any place you have used `strlen`, `strcpy`, `strcmp`, and especially if you did something like `strlen(encoding) % i` you are not well prepared.

If you do not have a good implementation of the encoder from lab A, consider the following advice. You may also borrow lab A solutions in C from other people (**only**) as preparation for lab 3, but of course you should say your implementation is based on (appropriate citation) C code from another source. In this instance only this is permitted and will cause no grade reduction! If, on the other hand, you do have a good low level implementation of the encoder from lab A, this task 0.C can be considered a "nop", and ignore the rest of task 0.C. That is, you are ready in this aspect due to a previously well done job! (As the saying goes "*Mi She Tarakh Be Erev Shabat, Yokhal Be Shabat*").

Low-Level Encoder Tips

A good low-level implementation of the encode is easy to transfer to assembly language. Good is according to the tips below:

- Note that you never need to "copy" a "string" in this task. Rather you can always maintain a pointer to its start, as do not need to modify such "strings". Therefore, you should not be copying any such, and certainly not use `strcpy`. See next tip.
- Recall that a "string" is simply an array of bytes, and that a pointer can be seen as a reference to the array, or any part thereof if it is advanced. So no need to use `strcmp` to detect command-line flags. For example, to do the output file case, can simply do in the loop on arguments:

```
char * OutFileName;
FILE * OutFile;
if(av[i][0] == '-' && av[i][1] == 'o') {
    /* Can actually be done in 1 instruction, e.g.:
       CMP word [eax], '-'+(256*'o') ; equivalently "-o" */
    OutFileName = (av[i])+2;
    OutFile = fopen(OutFileName, "w");
    if(Outfile == NULL) { /* error, print "cannot open file" and exit */ }
}
```

To find the start of the encoding string, same `(av[i])+2` as above works:

```
unsigned char * EncoderString, * CurrentEncodeP, EncodeByte;
if(av[i][0] == '+' && av[i][1] == 'V') {
    CurrentEncodeP = EncoderString = (av[i])+2;
    if(*EncoderString == 0) /* Error, null encoder string, exit */
}
/* And then, later on, after getting each character c: */
EncodeByte = (*CurrentEncodeP) - '0';
c += EncodeByte;
CurrentEncodeP++;
/* And then below wrap around to start at null termination */
if(*CurrentEncodeP == 0) { CurrentEncodeP = EncoderString; }
```

Task 1: Simplified Encoder In Assembly Language

Recall the encoder program from lab A (a long time ago!). We want to implement a simplified version of the encoder program in assembly using system calls. Please note again that your code (including the function `main`) should be **exclusively in assembly language**, and use no library functions. You may still call functions from `util.c` which we provide.

Overall, the encryption program should support the following command-line arguments:

Flag	Description
<code>+V{key}</code>	Designate Vigenere cipher encoding key
<code>-i{file}</code>	Get input from the given file
<code>-o{file}</code>	Direct output to the given file

The encoder will be a simplified version that reads a character (from stdin by default), encodes it according to the Vigenere cypher, and outputs it (to stdout by default). We do this in the 3 subtasks below. Note that you only need to support `+V` (no need to support `-V`). Also, you may encode only the lower case characters, still earning full credit (though a TA may ask you to encode upper case characters instead).

Task 1.A: Debug Printout in Assembly Language

This part achieves access to arguments in cdecl C calling convention, and acts like the debug printout of lab A. Except now debug is always on, so all command line arguments are printed to stderr. Write a function called `main` in assembly language that prints all the command line arguments to stdout, each on a separate line, using only direct system calls (that is, use the `write` system call, number **4**). You are supposed to use no library functions, but may use our `strlen` provided in `util.c`. The program should then exit "normally" using the `exit` system call (system call number **1**).

Task 1.B: Basic Encoder Version

In this section you are required to implement the encoder program from stdin to stdout (use the appropriate system calls for reading or writing). Extend the code from Task 1.A to do this. We suggest using another function called `encode` to do this. For simplicity we also suggest using global variables `Infile` and `Outfile` (appropriately initialized) rather than constants in the system calls, in order to speed up the next subtask. If you find it hard to do the entire encoder, at least support Vigenere cypher keys of length 1 for partial credit.

Task 1.C: Encoder Version With Input and Output Support

Add to your program the option of the following command line arguments:

- `-i{file}` — get input from the given file.
- `-o{file}` — direct output to the given file.

Recall from Task 0.C how to access the file name, to be used in the system call. Also, note that the system call is equivalent to `open` (except for the return value) rather than `fopen`.

Task 2: Attach Virus Program

Many computer viruses attach themselves to executable files that may be part of legitimate programs. If a user attempts to launch an infected program, the virus code is executed before the infected program code. The goal is to write a program that attaches its own code at the end of files in the current directory.

In this task you are required to write both in C and assembly language. Your C program should call `sys_getdents` in order to get all the entries in the current directory and print them, and when the `-a{prefix}` argument is provided should call assembly functions that will attach the executable code to the desired files.

Description

Print all the file names in the current directory.

Command-Line Argument

`-a{prefix}` — Attach the executable code (as discussed further below) at the end of each file in the current directory that begins with the given 1 character prefix. For every file changed you should print a message to the user, i.e., when printing the file name as a part of the file list add a print comment **"VIRUS ATTACHED"** next to the file name.

Guidelines

- Your program should use the `sys_getdents` [141] system call.
- The declarations of the `dirent` type constants can be found in the file `dirent.h`.
- Please note that the first argument for `getdents` is a file descriptor open for reading — it should be for the file `"."` that represents the current directory.
- In case of an error, the program should terminate with exit code **0x55**.
- To make things easier, you may assume that the entire directory data (returned by the `getdents` call) is less than **8192 bytes**.
- **Do not forget not to use any standard library functions!** Instead, in `util.h` and `util.c`, you can find few implementations for some helpful functions. You may use them.

Task 2.A: Directory Listing

Print all the filenames in the current directory to stdout. Note that this part is to be written in C (but with no standard library). If given the `-a` in the command line, print only the filenames for which the 1 character prefix of the file name matches the characters after the `-a`.

Task 2.B: Attaching the Virus

Now implement the `-a` option to actually attach the virus code.

Warning: You probably want to be very sure that the mechanism for selecting files works correctly at this point, e.g. you may not want the program to operate on your C source code files, etc. **Be careful not to destroy your own source code files!**

The following contains code you need to write in **assembly language**:

- I Starting assembly language implementation: begin with a label `code_start`.
- II Write a function `void infection()` that prints to the screen the message "`Hello, Infected File`". Note: this should be done using just **one** system call! If you have too many lines of code here then you are doing something wrong!
- III Write a function `void infector(char *)` that opens the file named in its argument, and adds the executable code from `code_start` to `code_end` after the end of that file, and closes the file. Note: this should be done using just a few system calls: **open** (for append), **write**, **close**, each using less than 10 lines of assembly code. Again, if your code is longer then you are doing something wrong!
- IV End infected and infector program part with a label `code_end`.

When the flag `-a{prefix}` is supplied, your program will call the `infection` and `infector` functions in order to add the executable code of infection to the end of each file with the given prefix. Also, this option will print out a comment such as "**VIRUS ATTACHED**" next to each file with the given prefix (this printout will be performed in the C program).

Note: It is recommended to open the file with the append option enabled (also need write of course). You may open for reading/writing rather than append, but then you will have to perform the `lseek` system call to the end of the file.

Note for assembly language implementation: The part of the code that is responsible for actual file handling (i.e. opening the file, adding the executable code of the infection, etc.) should be written in assembly language and done inside the file `start.s`. You can add the code after the end of the code for `system_call`. You can either call the `system_call` code (note that it uses C calling conventions, as until now you used it through function-calls from C), or re-use

part of it to do the system call yourself (shorter and simpler!). Also, it is a good idea to test your `infection()` function first, before proceeding to `infector()`.

Test your implementation on at least two files. You can use your previous lab solutions as input. Use the command `chmod u+wx {filename}` to give user write/execute permissions.

Task 2.C: Shortest Code Competition (Bonus Task)

Given spare time in the lab, the TA should hold a shortest assembly code competition for this part of the lab. Winner gets to brag about it to friends and neighbours, and in job interviews.

Submission

Task 1 and Task 2 are mandatory for this lab.

You are required to submit a zip file named `[your id].zip` that contains the following:

```
+ task1/
  - start.s
  - makefile
+ task2/
  - main.c
  - start.s
  - makefile
```